



广州联网科技

PG 系统定位原理及程序框架介绍



D-DWM-PG1.7/2.5/3.6/PLUS/4.6/4.9

定位原理及模块功能介绍

竭诚感谢您使用本公司的产品

本手册就产品的使用方法与安全事项进行说明

*熟读本手册，并在使用过程中注意安全。

*保留本手册，放在合适的地方以便随时查阅。

更新日期：2024 年 6 月



目 录

1、UWB 通讯原理	5
2、TWR 测距原理	6
2.1 DS-TWR 的测距	6
DS-TWR 测距通讯数据格式	8
2.2 HDS-TWR 的测距	10
HDS-TWR 测距通讯数据格式	11
3、定位原理	16
4、模块实现定位的框架	19
5、程序源码简介	21
5.1 硬件源码	21
5.1.1 DS-TWR 模式	22
5.1.2 HDS-TWR 模式	22
5.1.3 硬件源码其它细节	26
5.2 定位解算算法说明	28
5.3 上位机源码	32



安全注意

阅读本手册后，请妥善保管以便查阅。



这里展示的是注意事项和安全相关的重大内容，所以请一定要遵守，标志意思如下：



警告 在操作时违反本警告事项所示的内容，可能会导致人员死亡或重伤。



注意 在操作时违反本注意事项所示的内容，可能会导致人员负伤或造成物品损坏。



提醒 在操作时使您能正确使用产品时，所务必遵守的相关使用



1、UWB 通讯原理

UWB(Ultra Wideband),超宽带技术,它源于 20 世纪 60 年代兴起的脉冲通信技术。

从一般的通信体制来看,传统通信是利用一个高频载波来调制一个窄带信号,通信信号的实际占用带宽并不高。而 UWB 不同于传统的通信技术,它是通过发送和接收具有纳秒或微妙级以下的极窄脉冲来实现无线传输的。由于脉冲时间宽度极短,因此可以实现频谱上的超宽带,使用的带宽在 50MHz 以上。并且它不选用正弦载波,而是运用纳秒级的非正弦波窄脉冲传输数据,因而其所占的频谱区域很大,尽管使用无线通信,但其数据传输速率能够做到几百兆比特每秒以上。使用 UWB 技术可在非常宽的带宽上传输信号,美国联邦通信委员会(FCC)对 UWB 技术的规定为:在 3.1~10.6GHz 频段中占用 500MHz 以上的带宽。

PG 模块默认采用 DW1000 芯片通道 2 的信道,信道参数:中心频率为 3993.6MHz,频段为:3774 – 4243.2MHz,占用的带宽为 499.2MHz。

UWB 精准定位的关键优点有,低功耗、对信道衰落(如多径、非视距等信道)不敏感、抗干扰能力强、不会对同一环境下的别的设备造成干扰、穿透性较强(能在穿透一堵砖墙的环境进行精准定位),具有很高的精准定位准确度和精确度。

基于 UWB 的室内定位方案正在逐渐渗透飞机场、展览厅、办公楼、仓库、地下停车场、监狱、军事训练基地等需要使用准确的室内定位信息的应用。



2、TWR 测距原理

PG 系统采用 TWR 测距算法结合三边定位算法进行定位，所以大体上实现定位需要两个步骤：1、标签与基站进行测距得到距离值；2、利用这些距离值进行标签坐标的解算。下面讲解 TWR 测距有关的内容，PG 模块的测距分 DS-TWR 以及 HDS-TWR，下面介绍这两种测距的原理。

2.1 DS-TWR 的测距

DS-TWR 的测距原理图：

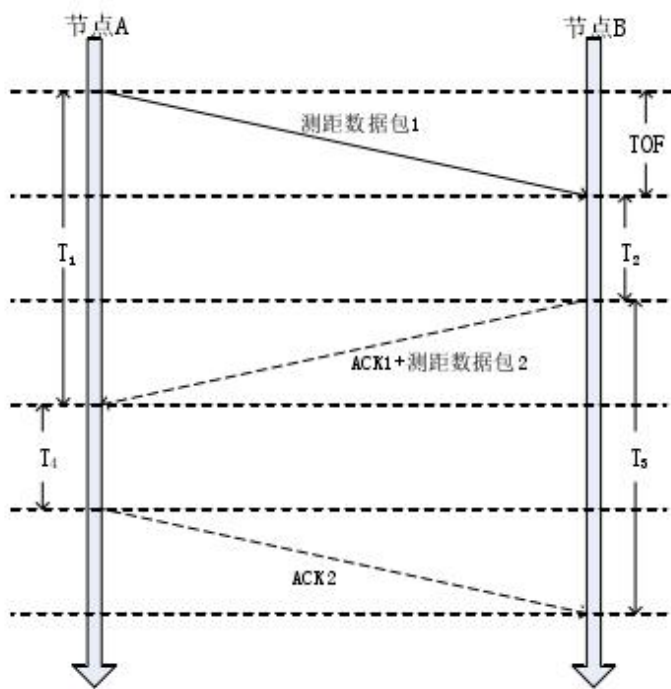


图 3-3 DS-TWR 测距原理

由原理图，我们可以得到：

$$T_1 = 2 \times TOF + T_2 \quad (2-1)$$

$$T_5 = 2 \times TOF + T_4 \quad (2-2)$$

$$T_1 + T_4 = T_2 + T_5 \quad (2-3)$$

将式子 2-1 和 2-2 左右两边相乘并移项整理可得：

$$T_1 * T_5 - T_2 * T_4 = 2 * TOF (T_5 + T_2) \quad (2-4)$$

联立 2-3 和 2-4 可得到信号飞行时间 TOF：

$$TOF = \frac{T_1 * T_5 - T_2 * T_4}{T_1 + T_2 + T_4 + T_5} \quad (2-5)$$



将 TOF 这个飞行时间和电磁波传输速率相乘即为节点 A、B 之间的距离。

下面分析一下这种测距方式的误差，假设节点 A 的钟差为 ℓ_A ，节点 B 的钟差为 ℓ_B ，且 $\ell_A = \ell_B$ 实际的 T_1 、 T_2 、 T_4 、 T_5 分别表示为 T_{t1} 、 T_{t2} 、 T_{t4} 、 T_{t5} ，实际的 TOF 为 T_f ，则有：

$$T_{t1} = (1 + \ell_a)T_1 = k_a T_1 \quad (2-6)$$

$$T_{t2} = (1 + \ell_b)T_2 = k_b T_2 \quad (2-7)$$

$$T_{t4} = (1 + \ell_a)T_4 = k_a T_4 \quad (2-8)$$

$$T_{t5} = (1 + \ell_b)T_5 = k_b T_5 \quad (2-9)$$

$$T_{t1} + T_{t4} = T_{t2} + T_{t5} \quad (2-10)$$

$$T_f = \frac{T_{t1} * T_{t5} - T_{t2} * T_{t4}}{T_{t1} + T_{t2} + T_{t4} + T_{t5}} \quad (2-11)$$

将 2-6、2-7、2-8、2-9、2-10 代入 2-11 可得：

$$T_f = \frac{k_a * k_b}{k_b} \frac{T_1 * T_5 - T_2 * T_4}{T_1 + T_2 + T_4 + T_5} = k_a * TOF \quad (2-12)$$

所以实际会造成的信号飞行时间误差为：

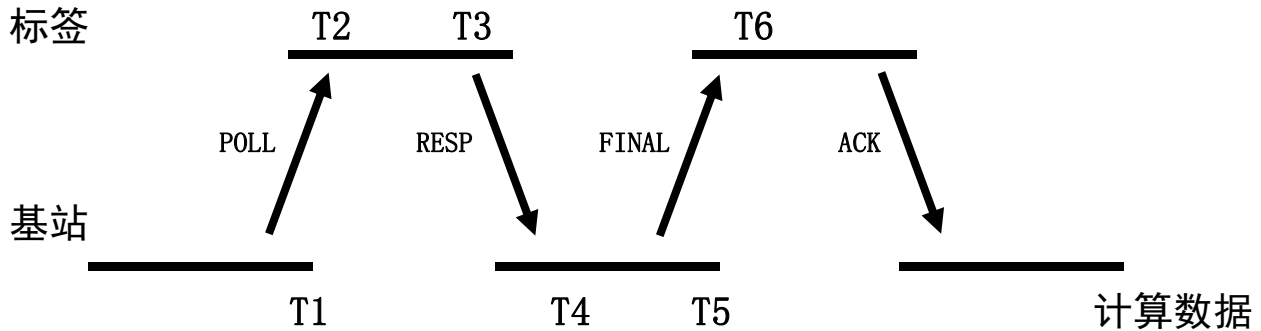
$$T_f - TOF = k_a * TOF - TOF = \ell_a * TOF \quad (2-13)$$

此时误差来源为 TOF 以及节点 A/B 的钟差，假设设备 A 和设备 B 的时钟精度都是 20ppm（很差），1ppm 为百万分之一，那么 K_a 和 K_b 分别是 0.99998 或者 1.00002。节点 A、B 相距 100m，电磁波的飞行时间是 333ns。则因为时钟引入的误差为 $20 * 333 * 10^{-15}$ 秒，导致测距误差为 2mm，可以忽略不计了。因此双边测距是最常采用的测距方式。



上述描述的 DS-TWR 测距方式是 A 设备主动访问，并且由 B 设备计算出距离。其中 T5 时间是又 T4+延时估算得出。

我们设置基站为主动端，标签为被动端，并牺牲一些时间去真实读取 T5 时间，这样可以提高精度以及提高系统稳定性。



时间戳	说明
T1	POLL 数据包 发出 基站-时间戳
T2	POLL 数据包 接收 标签-时间戳
T3	RESP 数据包 发出 标签-时间戳
T4	RESP 数据包 接收 基站-时间戳
T5	FINA 数据包 发出 基站-时间戳
T6	FINA 数据包 接收 标签-时间戳

DS-TWR 测距通讯数据格式

名称	地址	POLL	RESP	FINAL	ACK	A 基站呼叫次基站	次基站回传距离给主基站
发送方 ID	0 位	基站 ID	标签 ID	基站 ID	标签 ID	基站 ID	基站 ID
接收方 ID	1 位	标签 ID	基站 ID	标签 ID	基站 ID	基站 ID	基站 ID
帧数	2 位	数据帧号	数据帧号	数据帧号	数据帧号	数据帧号	数据帧号
命令	3 位	0XAB	0XBC	0XCD	0XDE	0XEF	0XFF
T1	4 位	1: 为测距 2: 为二维定位 3: 三维定位		T1	T1	需要测的标签 ID	计算成功标志
T1	5 位	上次定位成功标志		T1	T1		距离高位
T1	6 位	上次定位坐标 x 高八位		T1	T1		距离低位
T1	7 位	上次定位坐标 x 低八位		T1	T1		
T2	8 位	上次定位坐标 y 高八位	T2	T2	T2		



T2	9 位	上次定位坐标 y 低八位	T2	T2	T2		
T2	10 位	上次定位坐标 z 高八位	T2	T2	T2		
T2	11 位	上次定位坐标 z 低八位	T2	T2	T2		
T3	12 位	上次测距成功标志高八位			T3		
T3	13 位	上次测距成功标志低八位			T3		
T3	14 位	上次基站 A 测距高八位			T3		
T3	15 位	上次基站 A 测距低八位			T3		
T4	16 位	上次基站 B 测距高八位		T4	T4		
T4	17 位	上次基站 B 测距低八位		T4	T4		
T4	18 位	上次基站 C 测距高八位		T4	T4		
T4	19 位	上次基站 C 测距低八位		T4	T4		
T5	20 位	上次基站 D 测距高八位					
T5	21 位	上次基站 D 测距低八位					
T5	22 位	上次基站 E 测距高八位					
T5	23 位	上次基站 E 测距低八位					
T6	24 位	上次基站 F 测距高八位			T6		
T6	25 位	上次基站 F 测距低八位			T6		
T6	26 位	上次基站 G 测距高八位			T6		
T6	27 位	上次基站 G 测距低八位			T6		
数据透传	28 位	上次基站 H 测距高八位		使能接收数据透传	使能接收数据透传		
数据透传	29 位	上次基站 H 测距低八位		数据透传长度	数据透传长度		
数据透传	30 位	上次基站 I 测距高八位		预留数据透传	预留数据透传		
数据透传	31 位	上次基站 I 测距低八位		预留数据透传	预留数据透传		
数据透传	32 位	上次基站 J 测距高八位		预留数据透传	预留数据透传		



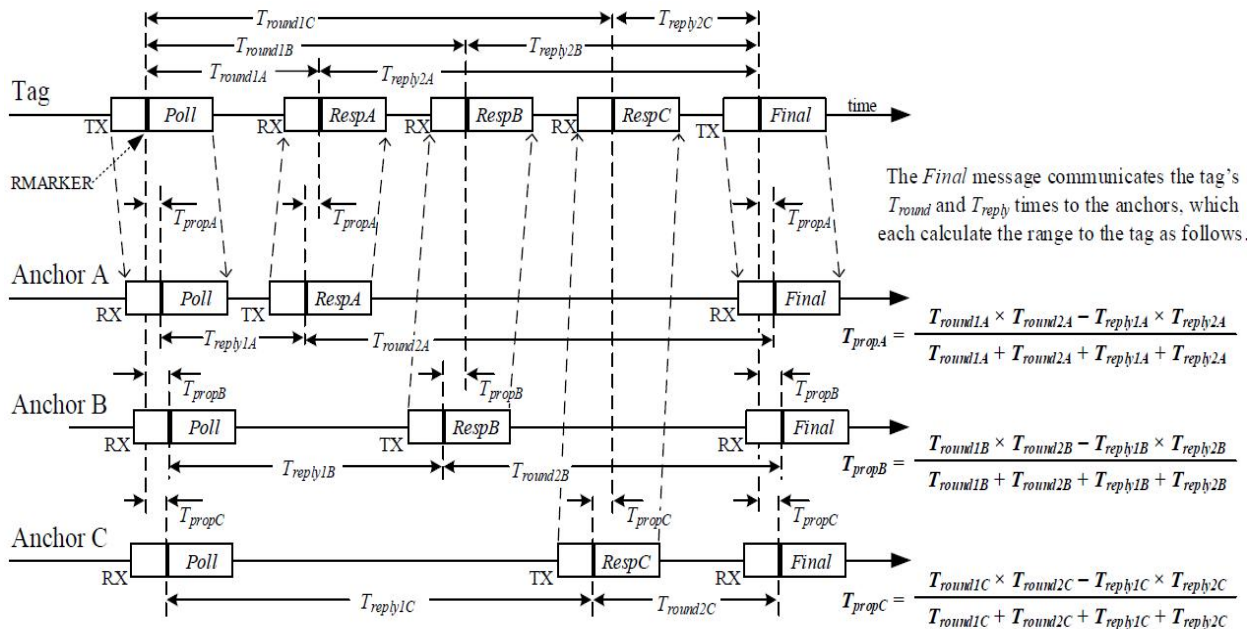
数据透传	33位	上次基站 J 测距低八位		预留数据透传	预留数据透传		
数据透传	34位	上次基站 K 测距高八位		预留数据透传	预留数据透传		
数据透传	35位	上次基站 K 测距低八位		预留数据透传	预留数据透传		
数据透传	36位	上次基站 L 测距高八位		预留数据透传	预留数据透传		
数据透传	37位	上次基站 L 测距低八位		预留数据透传	预留数据透传		
数据透传	38位	上次基站 M 测距高八位		预留数据透传	预留数据透传		
	39位	上次基站 M 测距低八位		预留数据透传	预留数据透传		
	40位	上次基站 N 测距高八位					
	41位	上次基站 N 测距低八位					
	42位	上次基站 O 测距高八位					
	43位	上次基站 O 测距低八位					
	44位	上次基站 P 测距高八位					
	45位	上次基站 P 测距低八位					
CRC 高	46位						
CRC 低	47位						

注：此表的每个数据包长度是不一致的，数据长度为有文字或英文描述的区域，其中 Final 包和 ACK 包的长度不定，会根据要透传的数据长度不同而有不同。



2.2 HDS-TWR 的测距

HDS-TWR 的测距原理图：



该算法的根本也是 DS-TWR，计算原理与 2.1 节的相同， T_{propA} 、 T_{propB} 、 T_{propC} 分别是标签到 A、B、C 基站的 UWB 信号飞行时间，但和 DS-TWR 不同的是该算法可以同时和几个基站同时测距，减少了测距需要的时间，从而缩短了标签定位的时间。

假设一个标签需要与 3 个基站测距，采用的是常规的 DS-TWR 测距流程，每个基站和标签测距需要 3 个信息包，总共就需要 9 个信息包。若采用 HDS-TWR，标签每一次与基站进行测距的时候，空中信息包只需要 5 个，减少了信息包的同时，也加快了定位的速度。

HDS-TWR 测距通讯数据格式

字节数	主基站 Inform 包	标签 Poll 包	基站 Resp 包	标签 Final 包	Request 包	Reply 包
0	主基站 ID	标签 ID	基站 ID	标签 ID	主基站 ID	次基站 ID
1	发送标签 ID	留空	标签 ID	留空	0xFF 或次基站 ID	主基站 ID
2	帧号	帧号	帧号	帧号	帧号	帧号
3	0xAA	0xAB	0xBC	0xCD	0xDE	0xEF
4	定位基站使能情况 8-16	使能接收数据透传	使能接收数据透传	RespA 接收时间戳	crc 高八位	计算成功标志
5	定位基站使能情况 0-7	数据透传字节总数	数据透传字节总数	RespA 接收时间戳	crc 低八位	测得的距离高八位
6	定位模式	预留数据透传	预留数据透传	RespA 接收		测得的距离



				时间戳		低八位
7	上次定位成功标志	预留数据透传	预留数据透传	RespA 接收时间戳		crc 高八位
8	上次定位坐标 x 高八位	预留数据透传	预留数据透传	RespB 接收时间戳		crc 低八位
9	上次定位坐标 x 低八位	预留数据透传	预留数据透传	RespB 接收时间戳		
10	上次定位坐标 y 高八位	预留数据透传	预留数据透传	RespB 接收时间戳		
11	上次定位坐标 y 低八位	预留数据透传	预留数据透传	RespB 接收时间戳		
12	上次定位坐标 z 高八位	预留数据透传	预留数据透传	RespC 接收时间戳		
13	上次定位坐标 z 低八位	预留数据透传	预留数据透传	RespC 接收时间戳		
14	上次测距成功标志高八位	预留数据透传	预留数据透传	RespC 接收时间戳		
15	上次测距成功标志低八位	预留数据透传	预留数据透传	RespC 接收时间戳		
16	上次基站 A 测距高八位	crc 高八位	crc 高八位	RespD 接收时间戳		
17	上次基站 A 测距低八位	crc 低八位	crc 低八位	RespD 接收时间戳		
18	上次基站 B 测距高八位			RespD 接收时间戳		
19	上次基站 B 测距低八位			RespD 接收时间戳		
20	上次基站 C 测距高八位			RespE 接收时间戳		
21	上次基站 C 测距低八位			RespE 接收时间戳		
22	上次基站 D 测距高八位			RespE 接收时间戳		
23	上次基站 D 测距低八位			RespE 接收时间戳		
24	上次基站 E 测距高八位			RespF 接收时间戳		
25	上次基站 E 测距低八位			RespF 接收时间戳		
26	上次基站 F 测距高八位			RespF 接收时间戳		
27	上次基站 F 测距低八位			RespF 接收时间戳		
28	上次基站 G 测距高八位			RespG 接收时间戳		
29	上次基站 G 测距低八位			RespG 接收时间戳		
30	上次基站 H 测距			RespG 接收		



	高八位			时间戳		
31	上次基站 H 测距 低八位			RespG 接收 时间戳		
32	上次基站 I 测距 高八位			RespH 接收 时间戳		
33	上次基站 I 测距 低八位			RespH 接收 时间戳		
34	上次基站 J 测距 高八位			RespH 接收 时间戳		
35	上次基站 J 测距 低八位			RespH 接收 时间戳		
36	上次基站 K 测距 高八位			Respl 接收 时间戳		
37	上次基站 K 测距 低八位			Respl 接收 时间戳		
38	上次基站 L 测距 高八位			Respl 接收 时间戳		
39	上次基站 L 测距 低八位			Respl 接收 时间戳		
40	上次基站 M 测 距高八位			RespJ 接收 时间戳		
41	上次基站 M 测 距低八位			RespJ 接收 时间戳		
42	上次基站 N 测距 高八位			RespJ 接收 时间戳		
43	上次基站 N 测距 低八位			RespJ 接收 时间戳		
44	上次基站 O 测距 高八位			RespK 接收 时间戳		
45	上次基站 O 测距 低八位			RespK 接收 时间戳		
46	上次基站 P 测距 高八位			RespK 接收 时间戳		
47	上次基站 P 测距 低八位			RespK 接收 时间戳		
48	crc 高八位			Respl 接收 时间戳		
49	crc 低八位			Respl 接收 时间戳		
50				Respl 接收 时间戳		
51				Respl 接收 时间戳		
52				RespM 接收 时间戳		
53				RespM 接收 时间戳		
54				RespM 接收		



				时间戳		
55				RespM 接收 时间戳		
56				RespN 接收 时间戳		
57				RespN 接收 时间戳		
58				RespN 接收 时间戳		
59				RespN 接收 时间戳		
60				RespO 接收 时间戳		
61				RespO 接收 时间戳		
62				RespO 接收 时间戳		
63				RespO 接收 时间戳		
64				RespP 接收 时间戳		
65				RespP 接收 时间戳		
66				RespP 接收 时间戳		
67				RespP 接收 时间戳		
68				Poll 包发送 时间戳		
69				Poll 包发送 时间戳		
70				Poll 包发送 时间戳		
71				Poll 包发送 时间戳		
72				Final 包估计 发送时间戳		
73				Final 包估计 发送时间戳		
74				Final 包估计 发送时间戳		
75				Final 包估计 发送时间戳		
76				crc 高八位		
77				crc 低八位		

注：此表的每个数据包长度是不一致的，数据长度为有文字或英文描述的区域。其中 Poll

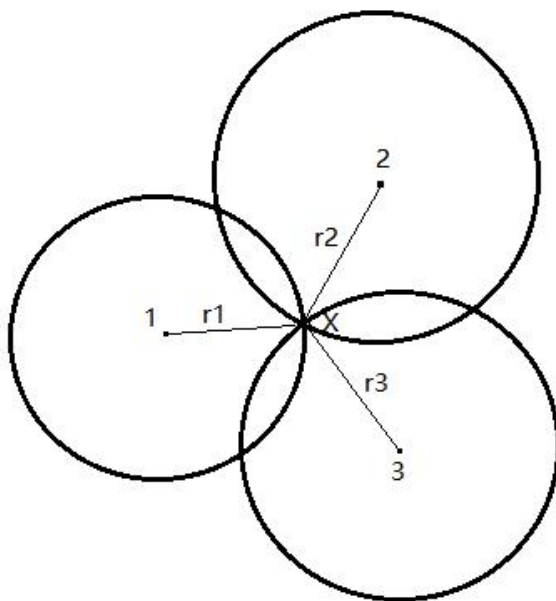


包和 Resp 包的长度不定，会根据要透传的数据长度不同而有不同。



3、定位原理

测距三边定位算法原理：平面上有三个不共线的基站 1、2、3 和一个标签 X，并已测出三个基站到标签 X 的距离分别为 r_1 、 r_2 、 r_3 ，则以三个基站坐标为圆心，三基站到未知终端距离为半径可以画出三个相交的圆，如下图，标签的坐标就是三个圆的相交点。



图中，1，2，3 为三个不同基站，三圆交点 X 为标签，假设基站 1,2,3 以及标签的坐标分别为 (x_1, y_1) ， (x_2, y_2) ， (x_3, y_3) 以及 (x, y) ，则可得到以下方程式：

$$\begin{cases} (x - x_1)^2 + (y - y_1)^2 = r_1^2 \\ (x - x_2)^2 + (y - y_2)^2 = r_2^2 \\ (x - x_3)^2 + (y - y_3)^2 = r_3^2 \end{cases} \quad (3-1)$$

将式子 3-26 线性化，可以得到线性方程组如下：

$$AX = B \quad (3-2)$$

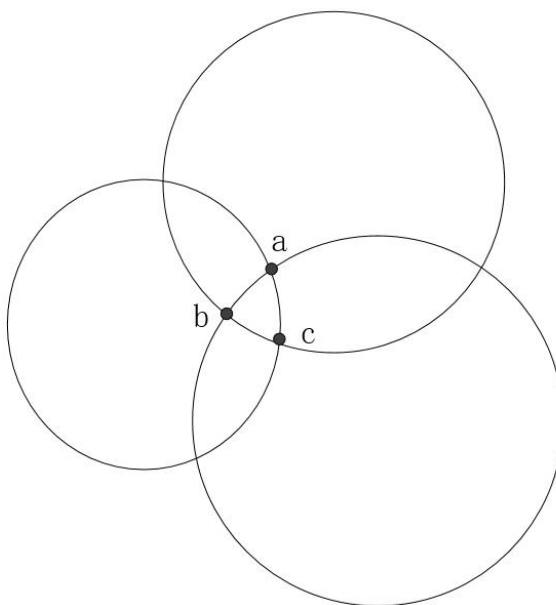
其中， $X = \begin{bmatrix} x \\ y \end{bmatrix}$ ， $A = \begin{bmatrix} 2(x_1 - x_3) & 2(y_1 - y_3) \\ 2(x_2 - x_3) & 2(y_2 - y_3) \end{bmatrix}$ ， $B = \begin{bmatrix} x_1^2 - x_3^2 + y_1^2 - y_3^2 + r_3^2 - r_1^2 \\ x_2^2 - x_3^2 + y_2^2 - y_3^2 + r_3^2 - r_2^2 \end{bmatrix}$ 。再解这

个方程组即可得到标签的二维坐标，但解这个方程组时需要矩阵 A 是可逆的，即需要： $(x_1 - x_3)(y_2 - y_3) - (y_1 - y_3)(x_2 - x_3) \neq 0$ 。这个式子表示了在实际应用时，三个基站不能摆放在同一直线上，如果确实存在这样的摆放情况，需要把这个情况的基站组合排除掉。

在基于上述的基础上，由于现实中有环境干扰或测距误差的存在，不太可能完全满足三



个基站测得的距离交于一点的情况。更多情况是例如下图：



因此，需要上述算法的精度在测距有误差的时候降低。可以通过采用最小二乘法的解算来减少误差，在（3-2）的基础上，使用最小二乘法解算 X ：

$$X = (A^T A)^{-1} A^T B \quad (3-3)$$

在 5.0 版本的更新中，更换算法为全质心算法解算，顾名思义，全质心算法就是求得三个圆交点 a、b、c 所成三角形的质心作为标签的坐标点。其计算出来的解均为实数解，具有比较高的精度和鲁棒性。

将（3-1）展开，可有：

$$\begin{Bmatrix} -2x_1 & -2y_1 & 1 \\ -2x_2 & -2y_2 & 1 \\ -2x_3 & -2y_3 & 1 \end{Bmatrix} \begin{Bmatrix} x \\ y \\ x^2 + y^2 \end{Bmatrix} = \begin{Bmatrix} r_1^2 - x_1^2 - y_1^2 \\ r_2^2 - x_2^2 - y_2^2 \\ r_3^2 - x_3^2 - y_3^2 \end{Bmatrix}$$

定义：

$$A = \begin{Bmatrix} -2x_1 & -2y_1 & 1 \\ -2x_2 & -2y_2 & 1 \\ -2x_3 & -2y_3 & 1 \end{Bmatrix}, \quad X = \begin{Bmatrix} x \\ y \\ x^2 + y^2 \end{Bmatrix}, \quad B = \begin{Bmatrix} r_1^2 - x_1^2 - y_1^2 \\ r_2^2 - x_2^2 - y_2^2 \\ r_3^2 - x_3^2 - y_3^2 \end{Bmatrix}$$

那么可以采用最小二乘法公式（3-3）解得 X 。

关于泰勒展开：

泰勒算法是一种基于泰勒级数展开的加权最小二乘估计算法，其主要思想是将非线性方程组在初始值处进行泰勒级数展开，忽略二次项后得到线性方程组，再通过迭代的形式求解。由于泰勒算法是一种迭代的算法，首先需要对待测标签的坐标有一个相对准确的初始估计值，然后在最小二次方差准则的前提下，连续迭代多次，直到达到预设的门限精度或迭代次数。

在获得初值 (x_k, y_k) 后，实际坐标可满足（3-4）公式：



$$\begin{cases} x = x_k + \Delta x \\ y = y_k + \Delta y \end{cases} \quad (3-4)$$

其中, $\Delta x, \Delta y$ 即估计坐标与真实坐标的差。

由 (3-1) 可知, 参与定位的基站测得标签的距离理想情况下满足 (3-5) 公式:

$$\sqrt{(x - x_i)^2 + (y - y_i)^2} = r_i \quad (3-5)$$

(x_i, y_i) 为各基站坐标。

计算出来的标签位置初值 (x_k, y_k) 代入到 (3-5) 后会有一定的误差, 各个基站等式都会有不同的误差。为了使各个基站的误差最小并得到一个更好的解算值, 可以通过对 (3-5) 做泰勒一次展开得到线性方程组以求解。

对于二元泰勒在 (x_k, y_k) 展开, 有:

$$r_i = f(x, y) \approx f(x_k, y_k) + (x - x_k)f'_x(x_k, y_k) + (y - y_k)f'_y(x_k, y_k)$$

将 $f(x, y)$ 代入到式 (3-5), 可得:

$$f'_x(x_k, y_k) = \frac{x_k - x_i}{\sqrt{(x_k - x_i)^2 + (y_k - y_i)^2}}$$

$$f'_y(x_k, y_k) = \frac{y_k - y_i}{\sqrt{(x_k - x_i)^2 + (y_k - y_i)^2}}$$

结合式 (3-4), 并拓展到所有基站写成矩阵形式, 可以通过最小二乘法计算出 $\Delta x, \Delta y$ 。

最小二乘法一般式有:

$$X = (A^T A)^{-1} A^T B$$

其中:

$$A = \begin{Bmatrix} \frac{x_k - x_1}{\sqrt{(x_k - x_1)^2 + (y_k - y_1)^2}} & \frac{y_k - y_1}{\sqrt{(x_k - x_1)^2 + (y_k - y_1)^2}} \\ \frac{x_k - x_2}{\sqrt{(x_k - x_2)^2 + (y_k - y_2)^2}} & \frac{y_k - y_2}{\sqrt{(x_k - x_2)^2 + (y_k - y_2)^2}} \\ \dots & \dots \\ \frac{x_k - x_i}{\sqrt{(x_k - x_i)^2 + (y_k - y_i)^2}} & \frac{y_k - y_i}{\sqrt{(x_k - x_i)^2 + (y_k - y_i)^2}} \end{Bmatrix}, \quad X = \begin{Bmatrix} \Delta x \\ \Delta y \end{Bmatrix}$$

$$B = \begin{Bmatrix} r_1 - \sqrt{(x_k - x_1)^2 + (y_k - y_1)^2} \\ r_2 - \sqrt{(x_k - x_2)^2 + (y_k - y_2)^2} \\ \dots \\ r_i - \sqrt{(x_k - x_i)^2 + (y_k - y_i)^2} \end{Bmatrix}$$

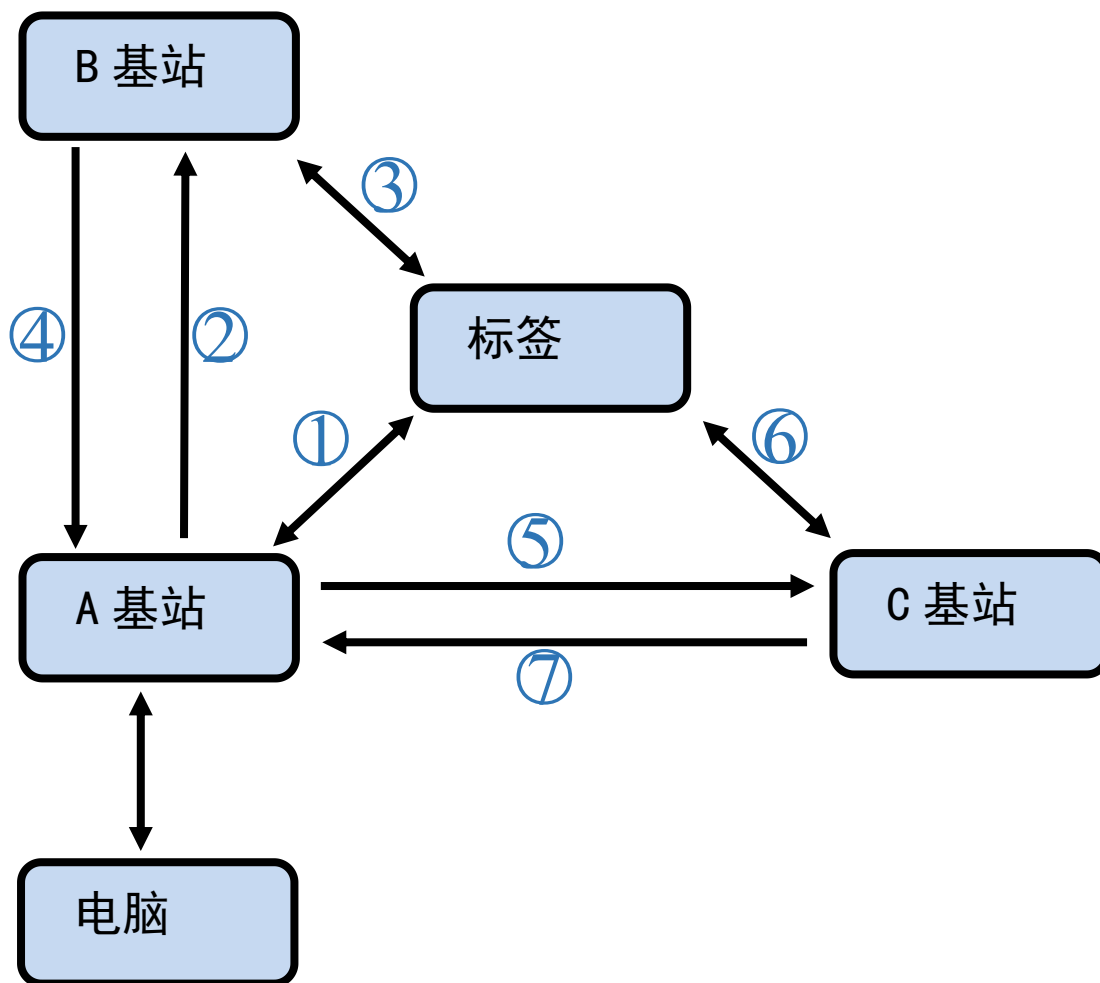
求解得到 $\Delta x, \Delta y$ 后, 可根据 $|\Delta x| + |\Delta y|$ 是否小于设置阈值来判断是否收敛完成。



4、模块实现定位的框架

采用 DS-TWR 时三基站一标签通讯图解：

系统以主动扫描标签方式构建，所有行为集中在主基站（A 基站）上发送命令并处理，并且有非常大的可操控性，所有的执行行为只需要操控配置主基站即可。更多基站以此类推。



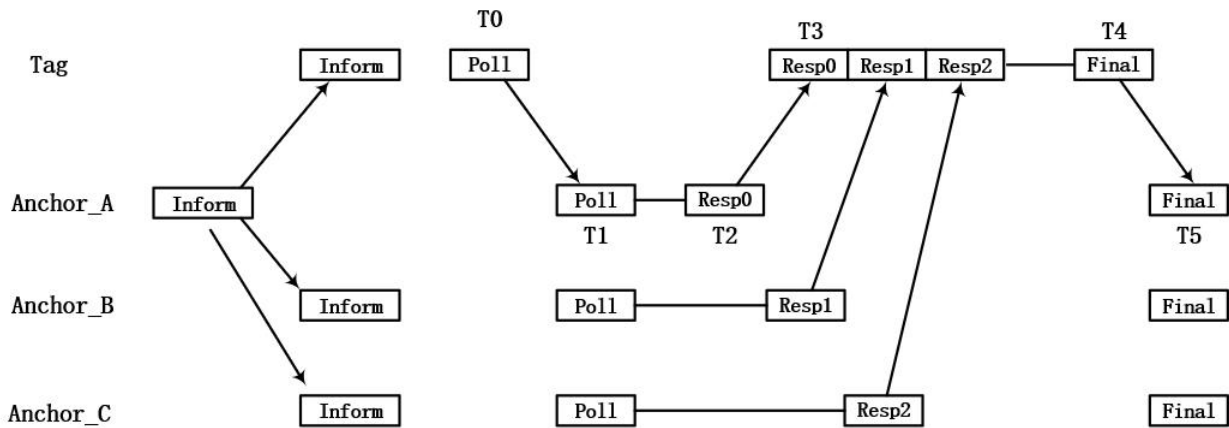
行为	主动端	被动端	说明
1	A 基站	标签	A 基站与标签测距
2	A 基站	B 基站	A 基站给 B 基站下达测距命令
3	B 基站	标签	B 基站与标签测距
4	B 基站	A 基站	B 基站将测距信息返回给 A 基站
5	A 基站	C 基站	A 基站给 C 基站下达测距命令
6	C 基站	标签	C 基站与标签测距
7	C 基站	A 基站	C 基站将测距信息返回给 A 基站



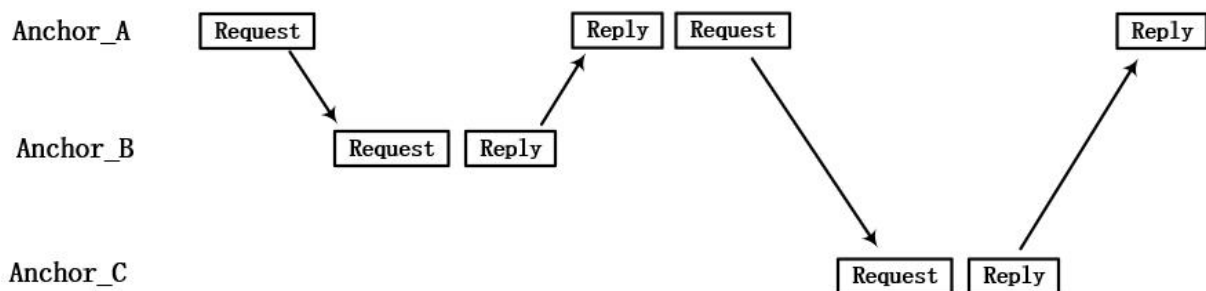
采用 HDS-TWR 时三基站一标签通讯图解：

系统以主基站为控制核心，主基站先发送测距开始信息，之后标签与基站开始测距。测距完成后，主基站发送请求包给次基站，然后所有次基站返回测距给主基站进行处理输出。更多基站也是以此类推。

测距流程图：



测距完成后，次基站返回测距值给主基站：



行为	主动端	被动端	说明
1	A 基站	标签、所有次基站	A 基站发送测距开始信息
2	标签	所有基站	标签发送 Poll 包给所有基站
3	A 基站	标签	A 基站发送响应包 Resp0
4	B 基站	标签	B 基站发送响应包 Resp1
5	C 基站	标签	C 基站发送响应包 Resp2
6	标签	所有基站	标签发送 Final 包给所有基站
7	A 基站	B 基站	A 基站请求 B 基站回传测距信息
8	B 基站	A 基站	B 基站回传测距信息给 A 基站
9	A 基站	C 基站	A 基站请求 C 基站回传测距信息
10	C 基站	A 基站	C 基站回传测距信息给 A 基站

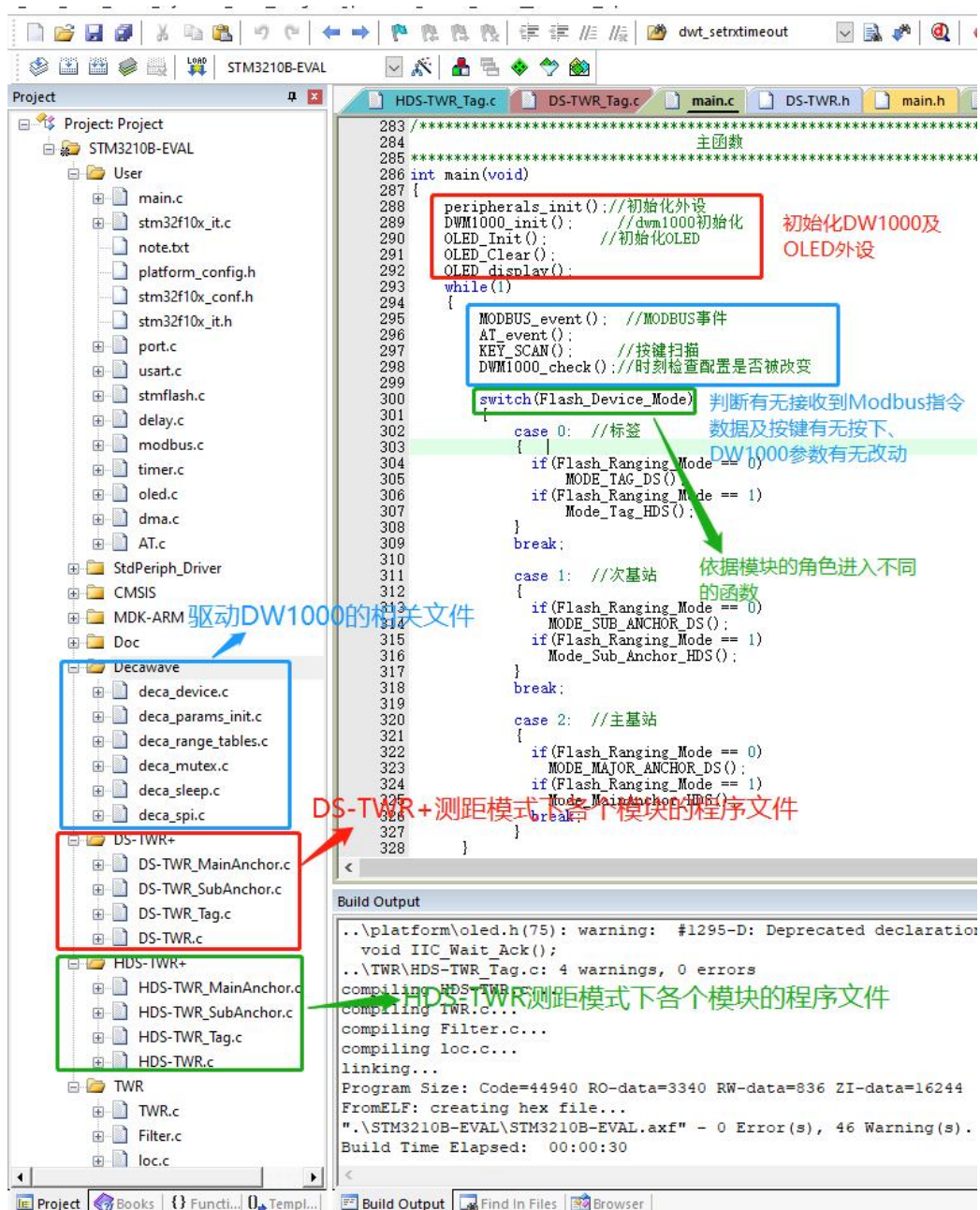


5、程序源码简介

程序源码都是基于以上原理进行设计的，下面对程序源码进行简介说明。

5.1 硬件源码

硬件程序项目文件解析：



两种不同测距方式的基站、次基站和标签的工作流程：



5.1.1 DS-TWR 模式

主基站：

定位开始后，主基站会对当前定位的标签进行 DS-Twr 测距，先发送 POLL 包给标签，收到标签的 RESP 包后发送 FINAL 包，之后接收标签回传的 ACK 包。接收后通知使能的每个次基站进行测距并得到每个次基站的测距值，获取到测距后进行定位解算，之后将标签测距结果及坐标等信息通过串口传输出去。

次基站：

处于一直监听的状态，收到主基站的开始测距的指令后发送 POLL 包给标签进行测距，收到标签的 RESP 包后发送 FINAL 包，之后接收标签回传的 ACK 包进行测距解算，解算后将测距值回传给基站。

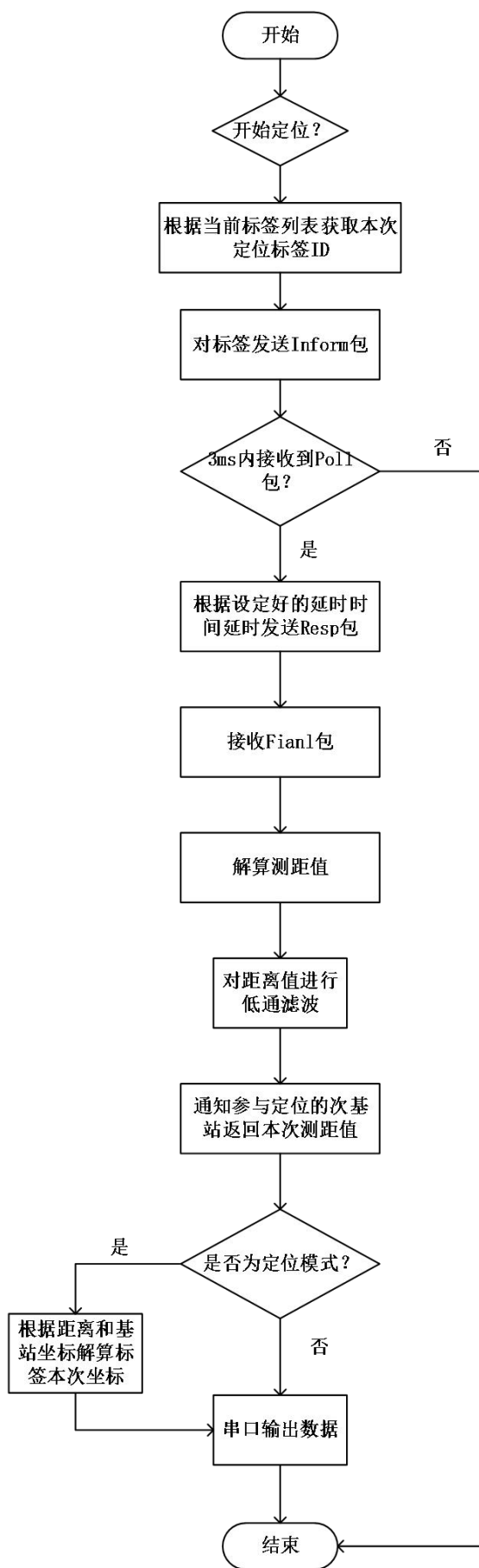
标签：

处于一直监听的状态，收到基站的测距的 POLL 包后，发送 RESP 包，之后收到基站的 FINAL 包后发送 ACK 包。

5.1.2 HDS-TWR 模式

主基站：

测距使能后，主基站会根据当前标签列表里面，选用当前测距到的标签作为 ID，对发一个包（Inform），并开启接收等待标签回应（Poll）包，接收到 Poll 包后，根据设定好的延时回送 Resp 包，并打开接收标签发送的 Final 包。主基站成功接收到了 Final 包后，会根据当前次基站使能情况，对参与测距的次基站依次发送距离回送请求包（Request 包），接收到次基站的 Reply 包来获取本次次基站的测距值。获取完所有的次基站测距后，如果测距则直接上报，定位则还要解算坐标后上传坐标。程序流程图如下：

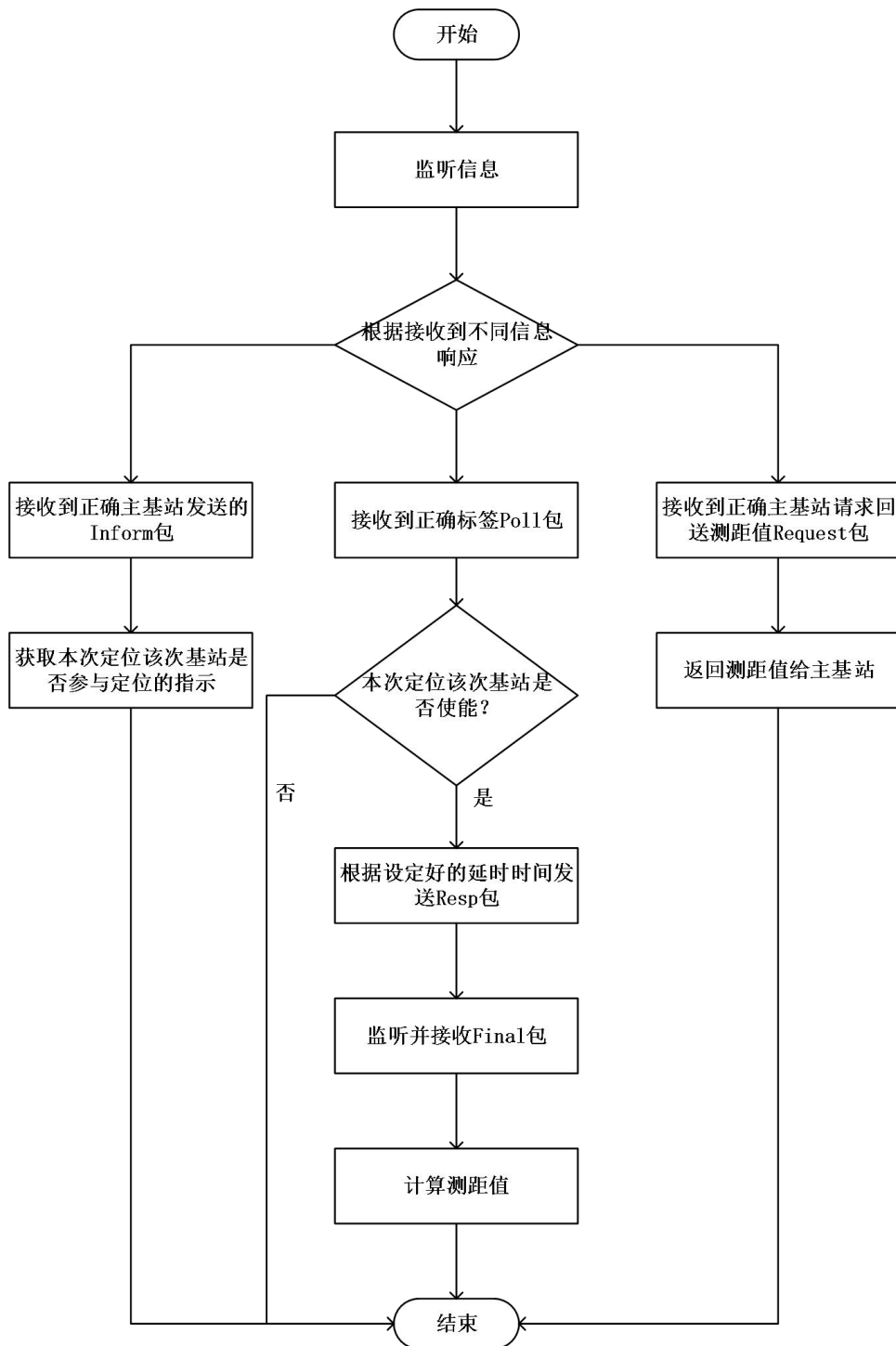


次基站:



处于一直监听的状态，根据接收到的不同包来做不同的动作。

会对主基站的 Inform 包有响应，它会根据这个包对应字节来知道这次该基站是否应该参与测距。如果是，则待会接收到标签的 Poll 包后会根据设定的延时时间发送 Resp 包，并等待接收 Final 包以计算测距值，不是则不发送；其次它还会对主基站发送的 Request 包有响应，回送本次测距 Reply 包。程序流程图如下：

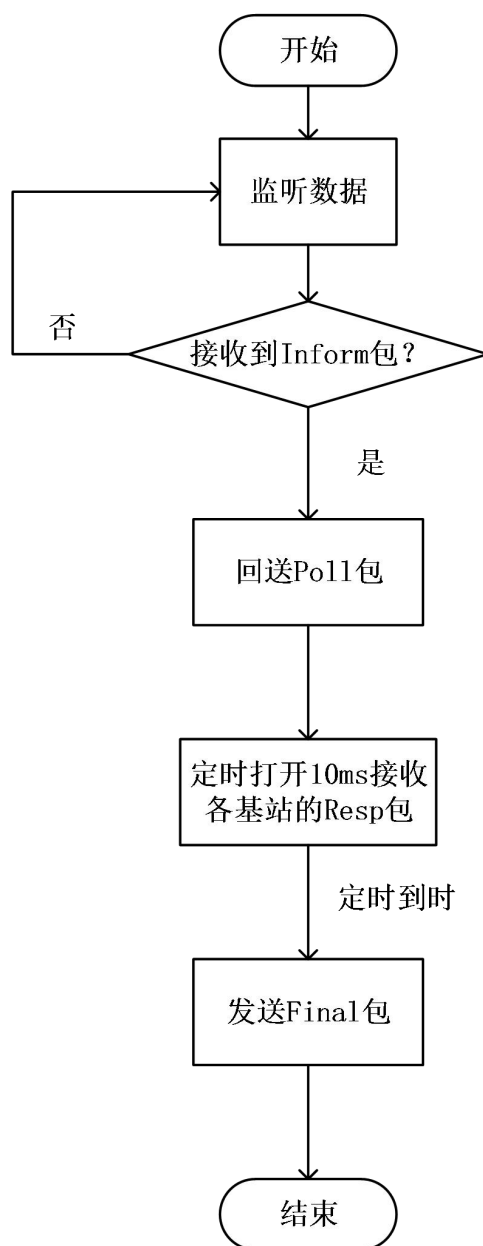


标签：

标签接收到正确的主基站发送的 Inform 包后，会发送 Poll 包，开启本次测距。它发送完



Poll 包后，会打开 10ms 左右的接收，接收 Resp 包。10ms 过去后，会估计本次的 Final 包发送时间，并写入到 Final 包中，延时发送 Final 包。





5.1.3 硬件源码其它细节

(1) 延时发送时间估计说明

由于 DW1000 的发送时间需要在发送完成后才会保存在本地的相关寄存器中，标签在发送 Final 包后，如果在不再多发一个包的情况下计算出测距值，就需要发送的时候采用延时发送，并估计发送时间，把这个估计的发送时间随 Final 包一并发送。

这个延时发送主要调用的 API 为 `dwt_setdelayedtrxtime(uint32 starttime)`。

对应的在 API 说明文档中，介绍如下：

5.16 `dwt_setdelayedtrxtime`

```
void dwt_setdelayedtrxtime (uint32 starttime) ;
```

This function sets a send time to use in delayed send or the time at which the receiver will turn on (a delayed receive). This function should be called to set the required send time before invoking the `dwt_starttx()` function (above) to initiate the transmission (in `DELAYED_TX` mode), or `dwt_rxenable()` (below) with `delayed` parameter set to 1.

Parameters:

type	name	description
uint32	starttime	The TX or RX start time. The 32-bit value is the high 32-bits of the system time value at which to send the message, or at which to turn on the receiver. The low order bit of this is ignored. This

© Decawave Ltd 2016

Version 2.4

Page 29 of 94

DW1000 Device Driver API Guide



		essentially sets the TX or RX time in units of approximately 8 ns. (or more precisely $512/(499.2e6*128)$ seconds) For transmission this is the raw transmit timestamp not including the antenna delay, which will be added. For reception it specifies the time to turn the receiver on.
--	--	--

调用该 API，会设定打开发送或接收的时间，参数 `starttime` 指的是在 `starttime` 的时间才会打开发送或接收，因此这个值是其计时器的值。比方说如果想要延时 2ms 才发送，那么 `starttime` 应该是系统时间再加上 2ms 的时间对应的寄存器值。

DW1000 内部计时器：DW1000 内部的计时器是以一个 40 位的寄存器来计时的。具体解释为：



7.2.8 Register file: 0x06 – System Time Counter

ID	Length (octets)	Type	Mnemonic	Description
0x06	5	RO	SYS_TIME	System Time Counter (40-bit)

Register map register file 0x06 is the System Time Counter register. System time and time stamps are designed to be based on the time units which are nominally at 64 GHz, or more precisely $499.2 \text{ MHz} \times 128$ which is 63.8976 GHz. In line with this when the DW1000 is in idle mode with the digital PLL enabled, the System Time Counter is incremented at a rate of 125 MHz in units of 512. The nine low-order bits of this register are thus always zero. The counter wrap period of the clock is then: $2^{40} \div (128 \times 499.2\text{e}6) = 17.2074$ seconds.

- Notes (a) On power up, before the digital PLL is enabled, the System Time Counter increments are still in units of 512 however the increment rate is half the external crystal frequency, (e.g. at 19.2 MHz for the 38.4 MHz crystal). The counter wrap period is then: $2^{31} \div 19.2\text{e}6 = 111.8481$ seconds.
- (b) In sleep modes the system time counter is disabled and this register is not updated.

每计数一个循环大约需要 17.2074s。而在 DW1000 正常工作的时候，它的计数低 9 位都是 0，每加一次 8 位的时间大约为 8ns，或者可以这么理解，计数的频率为 $128 \times 499.2\text{e}6$ 。那么，如果现实增加 100ms，那么需要计数的值就是

$$0.1 / (1/128 * 499.2\text{e}6) \approx 6,389,760,000$$

转换成 16 进制数，就是 0x17CDC0000。对于 starttime 的参数，将系统时间加上 0x17CDC00 就是延时 100ms 发送信息。以此类推，1ms 的计数值为 63897600，其 16 进制数为 0x3CF00

因此在使用延时发送的时候，一般的操作是首先读取系统时间的高 32 位或者读取上一次接收时间的高 32 位，然后与想要延时的值相加后作为 starttime 写入到函数中。然后再调用发送 API: `dwt_starttx(DWT_START_TX_DELAYED)`，即可进行延时发送。

注意：这个写入后，函数会自动为 starttime 左移 8 位作为延时发送的触发时间（还没有加上天线延时），因此 starttime 的值应该是计时器的高 32 位值。

DW1000 的发送时间是实际的寄存器的时间在加上发送天线延时时间的。写入的延时发送时间应该是 40 位的，所以需要将 starttime 左移 8 位后，再加上发送延时来作为其估计的发送时间。



5.2 定位解算算法说明

【5.0 版本前】

●二维定位算法

用的是 3 基站的矩阵直接解算的算法

筛选算法：

根据测距的基站数量进行判断，如果基站数量小于 3，则无法解算；如果基站数量大于 6，就每次排除测距值最大的基站直到基站数量为 6 个后才可以解算。

获取到所有参与解算的基站后，每三个进行组合。开始下一步判断。

三个基站首先判断三个基站能否组成一个三角形，用于排除三个基站同一直线的情况，然后再判断三个基站所成三角形的最大边长是否都会小于所有基站的测距值，如果是则认为该次测距，标签不在这 3 个三角形包围面内。最后，判断两两基站之间的测距圆是否相交，如果都相交，就可以进入解算坐标了。

●三维定位算法

用的是 4 基站的最小二乘法拟合解算。

根据距离方程可以写出 4 个距离方程式。然后和二维一样，每一行减去第一行后可以组成一个三阶行列式： $AX=B$ 。根据最小二乘法拟合的矩阵形式，最后解算出来是：

$$X = (A^T A)^{-1} A^T B$$

筛选算法：

筛选的目的是筛选到对角两高和两低的四个基站进行解算。

筛选步骤：

- a. 选取四个基站
- b. 判断水平坐标下能否形成三角形
- c. 判断立体情况下基站距离有没有都超过这四个基站组成四边形的边长
- d. 四个基站根据高度由高到低排序
- e. 判断次高的基站高度是否大于次低的基站高度大于至少 100cm
- f. 判断高与高基站水平坐标所成直线和低与低基站水平坐标所成直线是否相交的点是否在四个基站的内部



【5.0 版本后】

●二维定位算法

用的是 3 基站的全质心算法+泰勒算法

筛选算法：

根据测距的基站数量进行判断，如果基站数量小于 3，则无法解算；如果基站数量大于 6，则取测距值由小到大排序中，最小到第 6 小的 6 个基站来解算。

获取到所有参与解算的基站后，每三个进行组合。开始下一步判断。

三个基站首先判断三个基站能否组成一个三角形，用于排除三个基站同一直线的情况，然后再判断三个基站所成三角形的最大边长是否都会小于所有基站的测距值，如果是则认为该次测距，标签不在这 3 个三角形包围面内。最后，判断两两基站之间的测距圆是否相交，如果都相交，就可以进入解算坐标了。

所有满足筛选条件的 n 个基站组合可计算得到 n 个坐标点 (x_i, y_i) 。然后进行筛选，筛选方式如下：

- (1) 对 n 个坐标点求质心，也就是求 n 个坐标点的平均值，得到坐标 Q 。
- (2) 求 n 个坐标点到点 Q 的距离，并找到距离 Q 最大的坐标点 M 。
- (3) 去掉这个坐标点后再求这 $n-1$ 个坐标点的质心，得到坐标 P 。
- (4) 计算 Q 和 P 两个质心点的距离 d ，如果 d 大于阈值，则剔除 M 点后，重复开始 (1) - (4)。否则将第一次质心点 Q 输出，作为后面泰勒解算的初值。

得到了泰勒解算的初值后，根据公式进行迭代解算。如果泰勒解算的值的误差小于阈值或大于迭代次数即停止迭代并输出最终的坐标。

●三维定位算法

用的是最小二乘法拟合解算+泰勒算法

5.0 版本后不再判断和筛选基站，直接将基站全部进行最小二乘法的拟合。建议是基站布置在球形或趋于球形的位置以得到更好的精度。而且基站间的高度差最好和基站摆放的 x 和 y 方向上的最大差值接近。例如一个 $3*3$ 米的空间进行三维定位，那么基站间最大的高度差接近 3 米得到的效果会更好。

通过最小二乘法解算后，可得到泰勒解算的初值，然后根据公式进行迭代解算。如果泰勒解算的值的误差小于阈值或大于迭代次数即停止迭代并输出最终的坐标。



(2) 卡尔曼滤波说明

卡尔曼滤波通过对应的系统模型来从预测值和实测值之中自回归解算出更好的值。如果实际的情况与系统模型越接近，滤波效果会更好。

其滤波方式分为预测和校正两个步骤，总结来说卡尔曼滤波为 5 个方程（推导过程略）。

预测：

$$x_k = Ax_{k-1} + Bu_{k-1} \quad (1)$$

$$P_k = AP_{k-1}A^T + Q \quad (2)$$

更新：

$$K_k = P_k H^T (H P_k H^T + R)^{-1} \quad (3)$$

$$x_k = x_k + K_k (z_k - H x_k) \quad (4)$$

$$P_k = (I - K_k H) P_k \quad (5)$$

参数	说明
x_k	K 时刻的状态值
A	系统状态转移矩阵（建立的模型）
u_k	外界控制量
B	输入控制矩阵 代表输入控制量如何影响状态
P	误差矩阵协方差
Q	预测噪声协方差矩阵
R	测量噪声协方差矩阵
H	系统观测矩阵 代表将系统状态变量转换成测量的转换矩阵
K_k	K 时刻计算的卡尔曼增益
z_k	K 时刻的观测值

对于我们的定位系统使用的卡尔曼滤波模型，我们将系统简单假设为上一个时刻和下一个时刻的位置是没有发生变化的。且没有外部输入控制量， u_k 和 B 都是 0。观测矩阵 H 即为状态值，H 为单位矩阵 I。Q 和 R 假设过程中为恒定的一个值。

二维解算的状态值 x 应该是 $[x, y]$ 。由于 x 和 y 相互独立，那么为了简化计算过程，方便在单片机上计算，将状态值拆分为 x 和 y ，分别做两次卡尔曼滤波。由此，公式（1）到（5）在本模型中变为：

预测：



$$x_k = x_{k-1} \quad (6)$$

$$P_k = P_{k-1} + Q \quad (7)$$

更新:

$$K_k = P_k(P_k + R)^{-1} = \frac{P_k}{P_k + R} \quad (8)$$

$$x_k = x_k + K_k(z_k - x_k) \quad (9)$$

$$P_k = (1 - K_k)P_k \quad (10)$$

在编程的时候，可以设定误差矩阵和上一次状态值初始为 0，在几次迭代后就会到正常水平了。也可以先获取第一次的值作为初始值，这样前面几次的预测值会更加准确。Q 和 R 之间的关系会影响到滤波输出的值。如果 Q 远大于 R 元素，则预测噪声大，会更加相信使用观测值（本次计算出的坐标值），从实际来看就是系统的响应加快，但会没有那么平滑。反之则更相信预测值，卡尔曼滤波输出的结果更为平滑，但延迟就更大了。

卡尔曼原理讲解参考链接：<https://zhuanlan.zhihu.com/p/36745755>



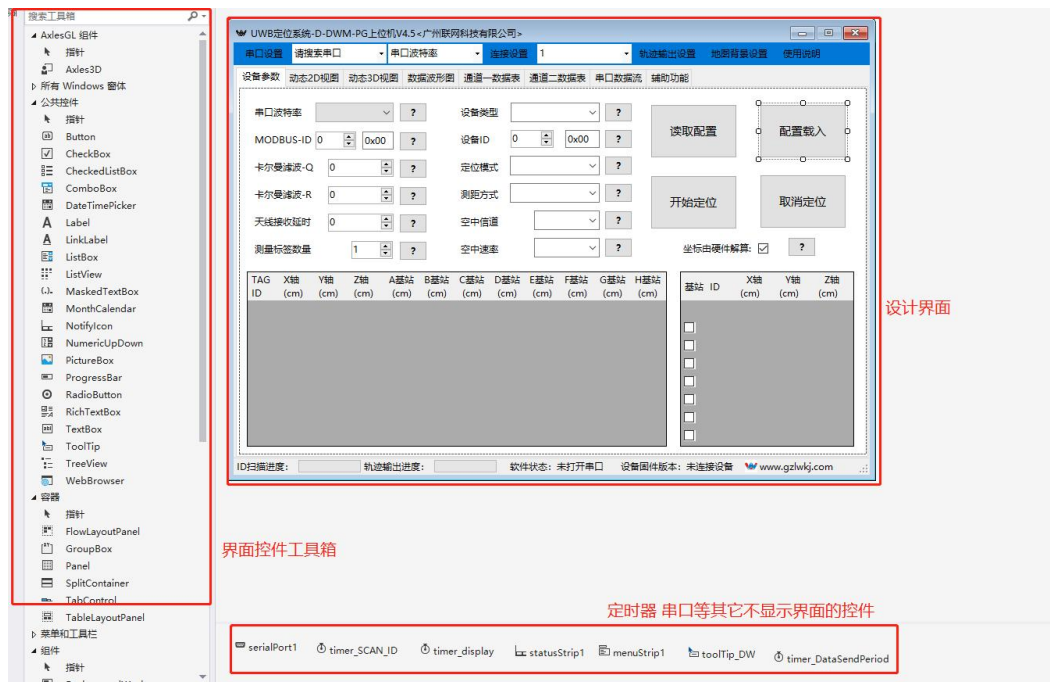
5.3 上位机源码

数据模型结构 串口数据 modbus 收发处理

上位机开发语言为 C#，使用 Winform 制作的上位机软件。

Winform 设计分为前台界面和后台程序两个部分。前台界面直接是使用拖拽控件来排列组合成界面，很简单。后台程序主要是处理界面动作触发后，对于数据的改动和计算算法。

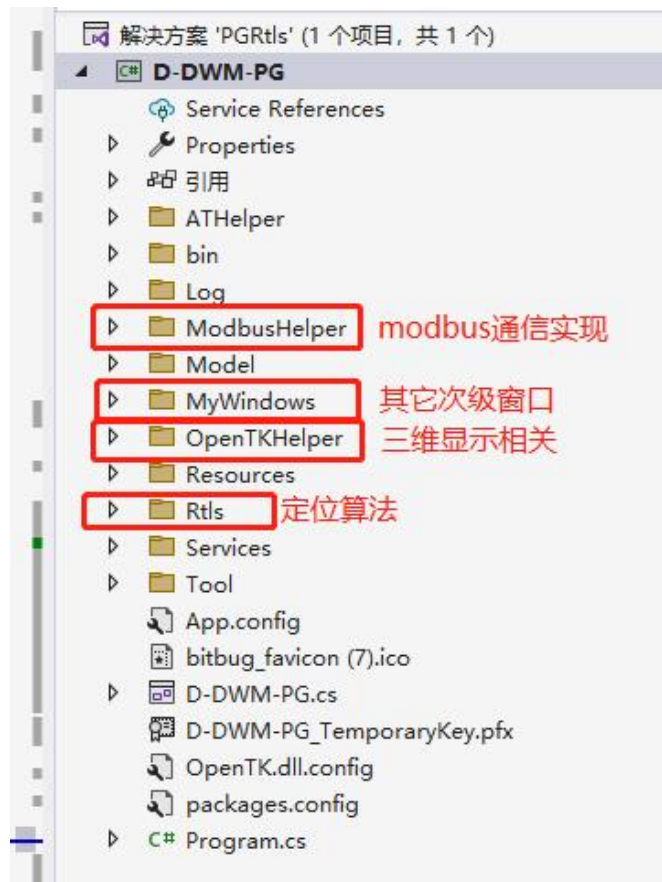
前台界面可以直接根据工具箱里的内容来排列组成：



后台程序：

该上位机主要的功能是通过与主基站串口连接，由串口收发的数据来实现控制定位系统和显示定位结果的功能。该上位机的设计功能主要是串口 Modbus 通信和数据传递画图显示。

后台程序结构：



Modbus 类:

Modbus 是一个单对单的协议，一般来说，需要主站发送一个信息，然后从站也因此回送相关的信息。因此，设计了一个 Modbus 类，里面包含了本次发送信息的 modbusID，寄存器地址，寄存器数量等相关 Modbus 需要的相关信息。在软件的主程序里，这个类的实例可以看作是一次固定的功能码之间信息的交互。比如，在主程序定义并实例了一个 Modbus 类为 modbus，那么发送前对应更改了这个类的信息后，接收也是会根据这个类里面的信息来尝试分析读取回送的数据。如果接收到与 modbus 里比如 ID，寄存器数量等信息相对应的数据，才会解读并传递到后面的函数当中。



```
class Modbus
{
    //ModbusID
    15 个引用
    public byte ModbusID { get; set; }
    //Modbus功能码
    17 个引用
    public byte FunctionCode { get; set; }
    //Modbus寄存器地址
    17 个引用
    public ushort Addr { get; set; }
    //Modbus寄存器数量
    22 个引用
    public ushort RegNum { get; set; }
    //Modbus寄存器值 06码会用到
    3 个引用
    public ushort RegValue { get; set; }

    /// <summary>
    /// Modbus类实例初始化
    /// </summary>
    /// <param name="ID">ModbusID</param>
    /// <param name="Func">Modbus功能码</param>
    /// <param name="addr">Modbus寄存器地址</param>
    /// <param name="regNum">Modbus寄存器数量</param>
    1 个引用
    public Modbus(byte ID, byte Func, ushort addr, ushort regNum)
    {
        ModbusID = ID;
        FunctionCode = Func;
        Addr = addr;
        RegNum = regNum;
    }
}
```

ModbusRTU 类:

ModbusRTU 类里实现了 Modbus03, 06, 10 码的发送和接收。它们的发送函数都会传入当前的 modbus 类, 根据 modbus 类里面的信息来组合需要发送的数据, 最后返回符合 Modbus 协议的字节数组来发送。具体的内容建议结合 Modbus 协议来一起看, 这里不再赘述。而接收的函数同理, 它会根据 modbus 类的信息来判断接收到的数据是否符合本次 Modbus 协议, 并且会根据解读的情况解读不同的状态。状态在 ModbusRTUState 类里。

ModbusRTUState:



```
26 个引用
class ModbusRTUState
{
    /// <summary>
    /// modbus接收状态指示
    /// </summary>
    26 个引用
    public enum ReceiveState
    {
        crcError = 0,           //crc校验错误
        IDError ,               //modbusID错误
        FunctionCodeError,      //modbus功能码错误
        AddrError,              //modbus寄存器地址错误
        RegNumError,            //modbus寄存器数量错误
        RegValueError,          //modbus寄存器值错误
        RecvOk                   //没问题
    }
}
```

比如 03 码接收:

```
/// <summary>
/// 03码接收
/// </summary>
/// <param name="buff">获取到的字节数组</param>
/// <param name="length">数组长度</param>
/// <param name="modbus"></param>
/// <returns>该数据modbus协议情况</returns>
1 个引用
public ModbusRTUState.ReceiveState Modbus03Recv(byte[] buff , int length, Modbus modbus)
{
    ushort crc = Crc16(buff, length - 2);
    if (crc != ((buff[length - 2] << 8) | buff[length - 1]))
        return ModbusRTUState.ReceiveState.crcError;
    if (buff[0] != modbus.ModbusID)
        return ModbusRTUState.ReceiveState.IDError;
    if (buff[1] != modbus.FunctionCode)
        return ModbusRTUState.ReceiveState.FunctionCodeError;
    if (buff[2] != modbus.RegNum * 2)
        return ModbusRTUState.ReceiveState.RegNumError;
    return ModbusRTUState.ReceiveState.RecvOk;
}
```

会根据不同 CRC，ModbusID，寄存器地址，寄存器数量对不对应的上来判断接收是否正确。

串口接收发送:

C#WInform 里可以直接使用其 Serial 类就可以构建一个串口通信的实例。通过使用该类的 Write 函数，即可发送数据。



```
#region 串口发送的函数
8 个引用
void Serial_SendData(byte[] buff)
{
    Task.Run(() => printf_data(buff, buff.Length, 0)); //显示发送信息

    try
    {
        serialPort1.Write(buff, 0, buff.Length);
    }
    catch
    {
        MessageBox.Show("串口通讯错误", "提示");
        return;
    }
}
#endregion
```

串口接收:

【旧版 4.5 前版本的接收】

串口接收可以通过给串口实例的接收数据事件绑定对应的接收事件，就可以实现对接收到的数据的处理。

```
1816 serialPort1.DataReceived += new SerialDataReceivedEventHandler(SerDataReceive);
```

通过绑定了事件后，每当串口接收到信息，就会进入到 SerDataReceive 这个函数中。

然后读取数据并根据协议进行判断。

```
/*
 * 串口接收中断函数
 * 1 个引用
 */
private void SerDataReceive(object sender, SerialDataReceivedEventArgs e) //串口接收数据程序
{
    if (serialPort1.IsOpen == false)
    {
        return;
    }

    //根据数据量和不同情况设定少量延时时间
    //延时是为了上位机能够接收全单片机发过来的数据
    if (Connect_State == ConnectState.Connecting)
    {
        Thread.Sleep((Int32)(Flag_BaudRate_Delay[Flag_BaudRate] * 30));
    }
    else
    {
        Thread.Sleep((Int32)(Flag_BaudRate_Delay[Flag_BaudRate] * 4));
    }
    //Thread.Sleep((Int32)(Flag_BaudRate_Delay[Flag_BaudRate] * 4));
    try
    {
        int Receivebuff_length;
        byte[] ReceiveByte; //串口数据接收数组
        ReceiveByte = new byte[serialPort1.BytesToRead];
        Receivebuff_length = serialPort1.Read(ReceiveByte, 0, ReceiveByte.Length);

        if (e.EventType == SerialData.Eof) //避免在0x1A报错
    }
}
```

【新版】

新版对数据的接收是统一先放在一个缓存区中保存下来，后台一直运行一个线程来判断



缓存区中的数据是否符合协议读取。设置接收到数据后，通过启动一个线程的开关量让该负责解析的线程开始工作解析。

```
1202
1283 2 个引用
1284 private void SerialDataReceive(object sender, SerialDataReceivedEventArgs e)
1285 {
1286     if (serialPort1.IsOpen == false)
1287     {
1288         serialPort1.Close();
1289         return;
1290     }
1291     int Byte_len = serialPort1.BytesToRead;
1292     if (Byte_len <= 0)
1293         return;
1294     byte[] Recv_byte = new byte[Byte_len];
1295     serialPort1.Read(Recv_byte, 0, Byte_len);
1296
1297     Monitor.Enter(recv_buffer);
1298     if(recv_buffer.Count > recv_buffer_max) //缓存超过字节数 先丢弃前面的字节
1299         recv_buffer.RemoveRange(0, recv_buffer_max);
1300
1301     recv_buffer.AddRange(Recv_byte); //存入缓存区
1302     Monitor.Exit(recv_buffer);
1303
1304     RecvThread_Are.Set();
1305     //Data_RecvHandler(ref sp_buffer);
1306 }
1307

1 /// <summary>
1 /// 对接收到的数据根据协议判断处理线程
1 /// </summary>
1 1 个引用
1 private void Data_RecvHandler()
1 {
1
1     while (true)
1     {
1         RecvThread_Are.WaitOne();
1         bool Is_Modbus = false;
1         if (recv_buffer.Count >= 4)
1         {
1             while (recv_buffer.Count >= 4)
1             {
1                 byte[] ReceiveByte = new byte[1];
1                 int ReceieveByte_length = -1;
1                 Is_Modbus = false;
1                 if (recv_buffer[0] == ModbusRTU.Instance.Modbus_com.ModbusID)
1                 {
1                     //接收到modbus数据
1                 }
1             }
1         }
1     }
1 }
```




数据模型：

数据模型的意义是让获取到的数据存放在后台，然后供给其它地方使用，避免繁琐的读取界面信息。其中需要建立的模型是基站、标签和接收信息模型。基站、标签模型主要是为了画图显示，它们里面存放了坐标、测距距离值等信息。定位数据从单片机上传到上位机后，读取分析协议后存放到对应的模型中，然后画图时候只需要根据模型里面对应的值来作图即可。下面是标签类的部分代码：

```
14 个引用
public class Tag
{
    //标签本次x坐标
    11 个引用
    public int x { get; set; }

    //标签本次y坐标
    11 个引用
    public int y { get; set; }

    //标签本次z坐标
    9 个引用
    public int z { get; set; }

    //标签自身ID
    10 个引用
    public int Id { get; set; }

    //标签本次定位是否解算成功
    7 个引用
    public bool CalSuccess { get; set; }

    //标签本次与其它八个基站测距值
    8 个引用
    public int[] Dist { get; set; }

    //标签本次与其它八个基站测距成功指示
    4 个引用
    public bool[] Dist_Success { get; set; }

    //标签上次x坐标
```

接收信息类 rxDiagnostic 类采用了数据绑定的做法。通过继承 INotifyPropertyChanged 类，每当实例中的属性发生了改变，将会自动通知到与其绑定的控件来更改显示的数据。



```
3 个引用
class rxDiagnostic:INotifyPropertyChanged
{
    //最大噪声
    private ushort _maxNoise;
    1 个引用
    public ushort maxNoise
    {
        get
        {
            return _maxNoise;
        }
        set
        {
            _maxNoise = value;
            OnPropertyChanged("maxNoise");
        }
    }

    //噪声均方根值
    private ushort _stdNoise;
    1 个引用
    public ushort stdNoise...
```

数据绑定:

```
/******
#region 设置数据绑定
1 个引用
private void DataBindingInit()
{
    Text_maxNoise.DataBindings.Add("Text", rxdiag, "maxNoise");
    Text_stdNoise.DataBindings.Add("Text", rxdiag, "stdNoise");
    Text_FpAmp1.DataBindings.Add("Text", rxdiag, "firstPathAmp1");
    Text_FpAmp2.DataBindings.Add("Text", rxdiag, "firstPathAmp2");
    Text_FpAmp3.DataBindings.Add("Text", rxdiag, "firstPathAmp3");
    Text_CIR.DataBindings.Add("Text", rxdiag, "maxGrowthCIR");
    Text_PreamCount.DataBindings.Add("Text", rxdiag, "rxPreamCount");
    Text_FP.DataBindings.Add("Text", rxdiag, "firstPath");
    Text_FPPower.DataBindings.Add("Text", rxdiag, "FPPower");
    Text_RxPower.DataBindings.Add("Text", rxdiag, "RxPower");
}
#endregion
```



广州联网科技有限公司

广东省广州市黄埔区莲花砚路1号

A栋902室

020-82000797

www.gzlwkj.com